

Project Report

on the topic

Implementation of Deep Learning Techniques in Demand Planning

at

O9 solutions.inc Bangalore.

A thesis

Submitted in partial fulfilment of the requirements for the award of the Degree of

INTEGRATED MASTERS OF SCIENCE IN MATHEMATICS AND COMPUTING



Under the guidance of

Dr. Vandana (Internal Guide)

Prepared by

Sibasish Padhy

(Roll. No. IMH/10023/17)

Birla Institute of Technology, Mesra,

May 2022.

APPROVAL OF THE GUIDE

Recommended that the thesis entitled “Implementation of Deep learning techniques-Demand Planning” prepared by Mr. Sibasish Padhy under my/our supervision and guidance be accepted as fulfilling this part of the requirements for the degree of Integrated Master of Science in Mathematics and Computing.

To the best of my/our knowledge, the contents of this thesis did not form a basis for the award of any previous degree to anyone else.

Date:

Dr.Vandana
Department of Mathematics
Birla Institute of Technology,Mesra

DECLARATION

I certify that

- a. The work contained in the thesis is original and has been done by myself under the general supervision of my supervisor(s).
- b. The work has not been submitted to any other Institute for any degree or diploma.
- c. I have followed the guidelines provided by the Institute in writing the thesis.
- d. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute e. Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references.
- f. Whenever I have quoted written materials from other sources, I have put them under quotation marks and given due credit to the sources by citing them and giving required details in the references.



Sibasish Padhy

(IMH/10023/17)

Birla Institute of Technology, Mesra

Date:



Department of Mathematics Birla Institute of Technology, Mesra, Ranchi -835215

THESIS APPROVAL CERTIFICATE

This is to certify that the work embodied in this thesis entitled "Implementation of Deep Learning techniques for Demand Planning" carried out by Mr. Sibasish Padhy (IMH/10023/2017) is approved for the degree of Integrated Master of Science in Mathematics and Computing of Birla Institute of Technology, Mesra, Ranchi.

Date:

Place:

Internal Examiner(s)
Name & Signature

External Examiner(s)
Name & Signature

Dr. Saral Kumar Jain
Head of The Department
Department of Mathematics
Birla Institute of Technology, Mesra

ABSTRACT

Demand Forecasting is a technical term used to describe Demand Planning for any product that is linked to or sold in a real market. It is critical for a company's supply chain management to be organised and planned. Throughout this endeavour, I used the word Actuals. The product is designated by the word SKU, which stands for stock keeping unit. For the hierarchical dataset I have used it is an aggregation of store id, item id, state id, on weekly level time horizon that operates like any other barcode while data is being processed. identified and handled as an individual. The inconsequential data that is being worked on has come from Walmart. This data is based on forecasting sales data for coming 4 weeks (4 weeks of future data) based on historical data of 8 weeks for 3 years of data. The forecast accuracy is then evaluated by comparing the actuals of the next 4 weeks with prediction of the 4 weeks.

Keywords: SKU, deep-learning, LSTM, RNN, hierarchical dataset, APE, forecast accuracy.

Acknowledgments

Firstly, I would like to express my sincere gratitude to my manager Mr. Nikhil Singh, Director and Head of Data Science at o9 solutions, Netherlands, Amsterdam, for giving me an opportunity to work as an intern in the talented pool of team of Data Science here at o9 solutions. I would also like to thank my second mentor Ravi Teja Ammanabrolu, for all his support and guidance.

Their continuous support during internship, patience, motivation, and immense knowledge helped me in all the time of research work and writing of this report. I could not have imagined having better advisors and mentors for my internship.

My sincere thanks also to Caner Turkseven, Data Scientist at o9 solutions, Amsterdam, Netherlands.

for helping me out in working issues, suggesting plausible approaches and encouraging me during tough times. I would also like to thank Dr.Vandana Guleria ,Department of Mathematics, Bit Mesra for her immense moral and constant support and guidance throughout the academic year and during the project.

Last but not the least, it would not have been possible to carry out such a task without moral support of my family.

O9 Solutions.Inc

Date: **9-05-2021**



Sibasish Padhy

Table of Contents

Abstract.....	2
Acknowledgement.....	3
1. Introduction.....	5
1.1What is Supply chain.....	5
1.2What is Supply Chain Management.....	5
1.3What is Demand Planning.....	5
1.4About the company.....	7
2. Literature Review.....	8
3. Background on RNN Architectures.....	8
3.1RNN ARCHITECTURES.....	9
3.2FORWARD PROPAGATION.....	9
3.3BACKWARD PROPAGATION.....	10
3.4BACKWARD TIME PROPAGATION.....	11
3.5 LONG SHORT-TERM MEMORY CELLS.....	12
3.6 GATED RECURRENT NEURAL NETWORKS.....	13
4.Methodology	
4.1 Flowchart.....	21
4.2 Summary of steps involved.....	22
4.3 Exploratory Data Analysis of the Dataset.....	22
4.4 Outlier Analysis and XYZ Analysis(Data Segmentation).....	22
4.5 Implementation of Different Deep Learning Algorithms for top 100 store item weekly sales demand combinations.....	22
4.6 Selecting the best and Implementation of the model on every time series.....	23
4.7 addition of some different features to improve upon the performance of the model.....	23
5.Case Study Analysis.....	23

6. Conclusions and Scope For Future Work.....	27
7.References.....	29

List of Figures

Figure 1. The illustration of different types of RNN architectures starting from sequence-to-sequence(top left),sequence-to-vector(top-right),vector-to-sequence(bottom-left),encoder-decoder architecture(bottom-right).The illustration is inspired from Geron et al(2019).....	10
Figure 2. The forward propagation of encoder-decoder Sequence-to-sequence architecture. The input-to-hidden, hidden-to-hidden, and hidden-to-output weights are denoted by $\mathbf{U}$, $\mathbf{W}$, and $\mathbf{V}$, respectively. The illustration is inspired from Goodfellow et al. (2016).For this thesis the context vector is repeated 4 times so for corresponding 4 context vectors we have 4 decoder setups.....	11
Figure 3 Backpropagation of the errors from output to lower layers. The flow also occurs across time steps for RNN.....	14
Figure 4: Structure of the long short-term memory (LSTM) cells. The illustration is inspired from G´eron (2019)	18
Figure 5: Structure of the gated recurrent neural network.....	20
Figure 6. Flow chart for the methodology applied for the implementation of Deep Learning techniques in demand Planning(Problem Statement)	21
Figure 7: Steps explaining the process of implementation of deep learning models for time series data.....	24
Figure 8: Forecast of 4 weeks(28 days) of prediction horizon using multilayer encoder-decoder LSTM for a particular store_item combination.....	24
Figure 9: forecasts for sku FOODS_1_001_TX_1_evaluation taken at weekly time horizon...	25
Figure 10: .forecasts for sku FOODS_1_001_CA_2 evaluation taken at weekly level.....	26

List of Tables

Table1: Table for every iteration along with the iteration description and accuracy.....27

Table2: Evaluation for all 30490 skus28

Table 3: Table for every iteration on the best model for all 30490 store_item combinations on the dataset.The iteration marked in yellow shows the best accurcacy obtained of all the iterations.
.....28

Table 4: THE BEST OPTIMAL HYPERPARAMETERS USED For Encoder Decoder GRU model.....31

1.Introduction

1.1 What is supply Chain

A supply chain is made up of all parties that are directly or indirectly engaged in fulfilling a client request. The supply chain includes not only the manufacturers and suppliers, but also transporters, Warehouses , retailers and even customers themselves.(Sunil Chopra and Peter Meindil).

1.2 What is supply Chain Management

A Supply chain Management refers to the various kinds of processes and techniques used to handle the entire production flow of goods and services from the raw components to way up delivering the finished product to the consumer.

A supply chain plays a pivotal part in regulating businesses in companies and much required for the company's success and customer satisfaction.

1.3 What is Demand Planning and Demand Forecasting

Demand Planning involves analysing sales along with consumer trends, historical sales and seasonality data to optimize the business's ability to meet customer demand in the most efficient way possible. To achieve this goal, demand planning combines sales forecasting, supply chain management and inventory Management(Demand planning :What is and why is it important : Abby Jenkins)

Demand forecasting is a subset of demand planning that analyses internal and external data to forecast sales.

Forecasts typically cover the next 18 to 24 months, however the prediction duration varies by product and sector.(Abby Jenkins et .al)

A key part of the supply Chain Planning involves demand planning and the associated demand Forecasting process. Because most production systems cannot respond to customer demand in real time, some estimate, or forecast, of future demand is essential in order to make efficient and successful operating plans. (James D.Blocher et al)

Blocher et al also discusses that since most of the production systems can't respond to consumer demand instantaneously, some estimate, or forecast, of future demand is required so that the efficient and effective operational plans can be made. Quantities like labour capacity ,process and plant are all set based on the forecast of future demand.

Longer-term aggregate forecasts would be used to guide capacity planning and facility decisions.

Forecasts are also required for efficient inventory level planning, which in practise necessitates shorter-term disaggregated forecasts because specific components, parts, and end-items must be supplied to meet current customer demand. Forecasts can also be used in finance, which requires an understanding of the appropriate amounts of investment in plant, equipment, and inventories.

Forecasting should be done primarily for end-items. When the final item's production quantities have been determined. When production quantities for the end item have been determined, component demand can be computed based on the plan of production of the end item and knowledge of the bill of materials (BOM). (Blocher et al.).

1.4 About The Company

o9 Solutions is a major supplier of an Augmented Intelligence (AI) platform that changes the digital supply chain's planning and decision-making capabilities. Clients of o9 come from a wide range of industries, including consumer goods, manufacturing, and retail. o9 Solutions helps organisations achieve new levels of productivity, speed, and expertise in decision making by delivering solutions that use modern AI technology to provide users with superior visibility, predictive and prescriptive insight, and cross-functional collaboration.

o9 was created in 2009 by Sanjiv Sidhu and Chakri Gottemukkala, two software entrepreneurs with a track record of creating value for multinational enterprises by offering creative planning solutions. In the early 1990s and early 2000s, Sanjiv was the founder and CEO of i2 Technologies, a firm that pioneered Advanced Planning and Supply Chain management software. Chakri was a vital contributor in a range of roles at i2, including product development, sales, and strategy, and he has a passion for building world-class solutions that address complicated issues.

Did you know o9 stands for **o**ptimization to the highest singular number (**9**)?
Oh and by the way, we are founded in 20**09**.

The above snapshot gives the perfect reference and justification for the pool of talent at o9.

The o9 Digital Brain provides capabilities for best market supply chain predictions and also for all the investments in the digital brain. The o9 performs all the capabilities for all the solutions via predictive modelling and forecasting using state of art Deep learning techniques and Machine learning techniques for better forecast.

My job as a data science intern was to work upon the timeseries datasets based on demand planning and to implement state-of-the-art deep learning algorithms like LSTM and GRU for predicting the sales for next 4 weeks, i.e. forecasting sales.

2. LITERATURE REVIEW

Binru Zhang, Yulian Pu, Yuanyuan Wang, have discussed in their article on Forecasting Hotel Accommodation Demand Based on LSTM Model Incorporating Internet Search Index[1] about how the uncertainty of the passenger flow during the tourist season makes the decision-making of relevant departments into a dilemma, either overestimating or underestimating of the passenger flow will

result in unnecessary waste of resources in tourism-related industries and non linear prediction algorithms like The deep learning RNN algorithm especially LSTM help in overcoming these non linear fluctuations by forecasting the tourist demand by forecasting near values to the actual tourist demand and showing a better accuracy score then its counterpart linear prediction algorithms like ARIMA,SARIMA

Zixin Dou , Yanming Sun ,Yuan Zhang, Tao Wang , Chuliang Wu and Shiqi Fan have discussed in their article :Regional Manufacturing Industry Demand Forecasting: A Deep Learning Approach[2] about the their experimentation of different prediction algorithms on demand forecasting of the manufacturing industry in GD which has become a national strategic industrial base, and advanced manufacturing is the leading industry with a large scale and a complete system. Deep learning algorithms like NN and LSTM are compared with other algorithms like RF(Random Forests),SVM(Support Vector Machines),AR(Auto Regressive processes) and BP. The accuracy of forecast(predictions) for LSTMS was found out to be better than other machine learning and Time series algorithms

Bahrudin I Hrnjica and Ali DANANDEH MEHR, have discussed in their article :Energy Demand Forecasting have discussed in their paper: Energy Demand Forecasting Using Deep Learning[3] about machine learning algorithms ,their structure, Artificial neural network and deep learning algorithms in detail.

They have applied autoencoder LSTM neural network for prediction of electricity consumption in the northern part of Nicosia during the 2011–2016. Nicosia, the capital city of Cyprus, has a typical Mediterranean climate with an annual average electricity consumption about 4000 MWh in its northern part and about 6000 MWh in the southern part. The series was decomposed into its 3 main components : Seasonality, trend, and noise. And this basis of its seasonality at yearly, weekly, and monthly level. its forecast was predicted and accuracy was evaluated this accuracy is being then compared with the forecast accuracy of autoencoder Lstm where it proved to be better. This Deep Learning model is then provided as a solution to the smart cities.

3. Background on RNN Neural Networks including LSTM and GRU.

We present a full mathematical explanation for recurrent neural networks in this section (RNN). We intend to offer context for RNN before presenting our methods in the next chapter. RNN is a type of neural network that can process input in a sequential manner (Rumelhart et al.,1986). Furthermore, unlike multilayer perceptrons (MLP), which can only process fixed-size inputs, RNN may handle arbitrary lengths of inputs. RNN has been used effectively in a variety of applications. Speech recognition, sentiment analysis, and picture captioning are examples of such issues (Graves et al., 2013;Vinyals et al., 2015; X. Wang et al., 2016).

In Section 3.1, we outline the several prevalent RNN designs, followed by the technique of Section 3.2 which discusses the transition from input to output, while Section 3.3 discusses RNN training and 4.4. Section 4.5 and Section 4.6 discusses the LSTM and GRU cells that address the short-term memory problem.

3.1 RNN architectures

Because RNN are capable of processing a succession of inputs and producing a single or a sequence of outputs, different RNN designs are employed for different reasons. As shown in Figure 4.1, this section discusses four possible RNN architectures: *sequence-to-sequence*, *sequence-to-vector*, *vector-to-sequence*, and *encoder-decoder*.

The *sequence-to-sequence* architecture (see top-left in Figure 3.1) takes a sequence of inputs and produces a succession of outputs. According to G'eron (2019), *sequence-to-sequence* architecture may be used for time-series prediction. RNN processes the prior observations $t = 1, 2, \dots, T$ at each time step T and outputs predictions for the following N time steps $t = T + 1, \dots, T + N$. As a result, error gradients emerge from the outputs at all time steps (G'eron, 2019).

On the contrary, the *sequence-to-vector* (see top-right of Figure) transformation excludes all intermediate outputs. At the end of the sequence, it just generates a single output. As a result, the error gradients only flow from the most recent time step (G'eron, 2019). This architecture may be utilised for a variety of purposes like Analysis of sentiment, such as categorising reviews as positive or negative (X. Wang et al., 2016).

The *vector-to-sequence* (see bottom-left in Figure 3.1) architecture processes the same input multiple times and generates a sequence of outputs. Goodfellow et al. (2016) discuss that *vector to-sequence architecture* can process a single image and generate sequence of words that describe the image.

Finally, the *encoder-decoder* design (shown in the bottom-right corner of Figure 3.1) may generate outputs of varied lengths (Goodfellow et al., 2016). As in a sequence-to-sequence design, the output length is not always the same as the input length. As a result, the encoder-decoder architecture may be utilised for Problems with machine translation (G'eron, 2019; Goodfellow et al., 2016). In this design, the encoder is in charge of encoding a sentence as a vector, while the decoder is in charge of translating the phrase to another language. G'eron et al., 2019; Goodfellow et al., 2016) G'eron (2019) contends that encoder-decoder architecture is superior to sequence-to-sequence architecture for machine translation difficulties because whole-word translation sentence should be waited to correct for different word orders across languages.

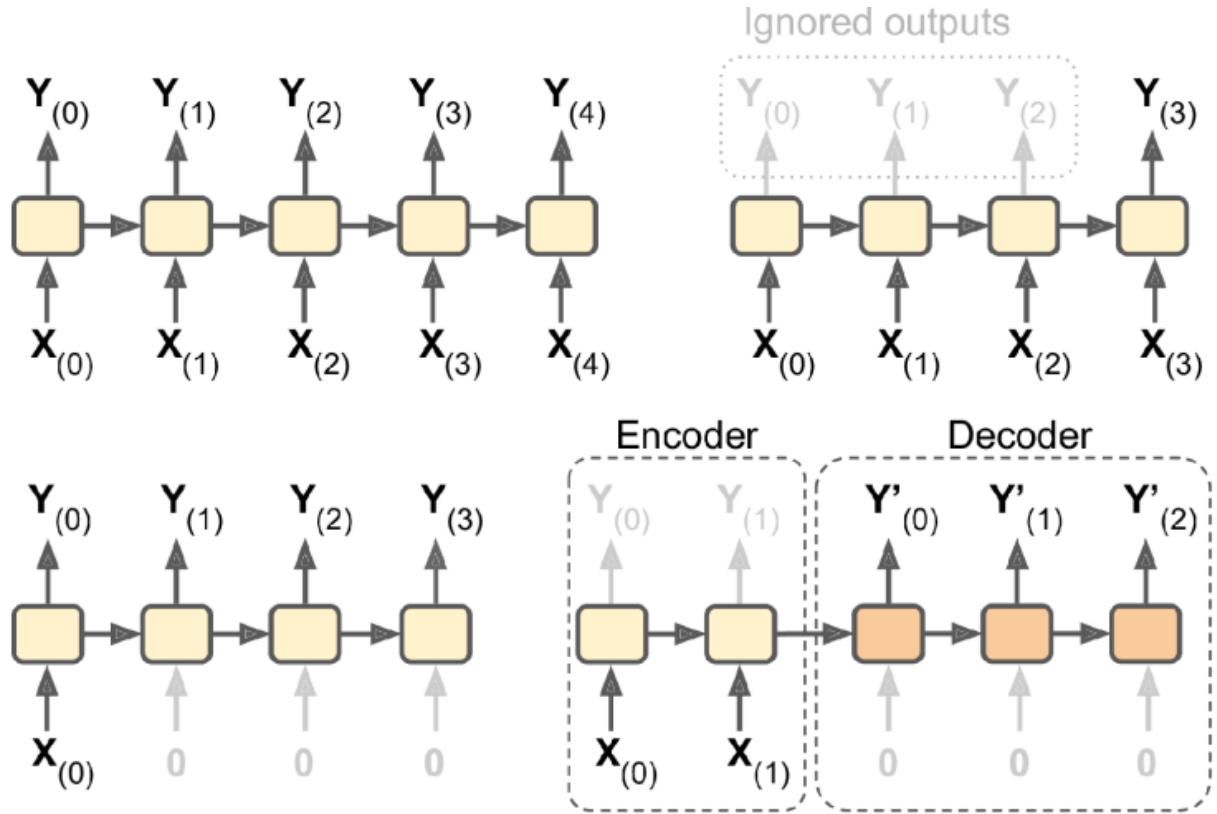


Figure 3.1 The illustration of different types of RNN architectures starting from sequence-to-sequence (top left), sequence-to-vector (top-right), vector-to-sequence (bottom-left), encoder-decoder architecture (bottom-right). The illustration is inspired from Geron et al (2019).

3.2 Forward Propagation

In this thesis, we model the probability of sales demand for every store_item_combination website and making a prediction about the sales in the next four weeks using the RNN algorithm. Since we use all of the historical weekly sales demand sequences of all the store_item combinations and our target is a univariate sales demand variable for all the SKUS, we utilize the sequence-to-sequence Encoder-Decoder architecture. The sequence-to-sequence encoder decoder architecture is illustrated in Figure 3.2.

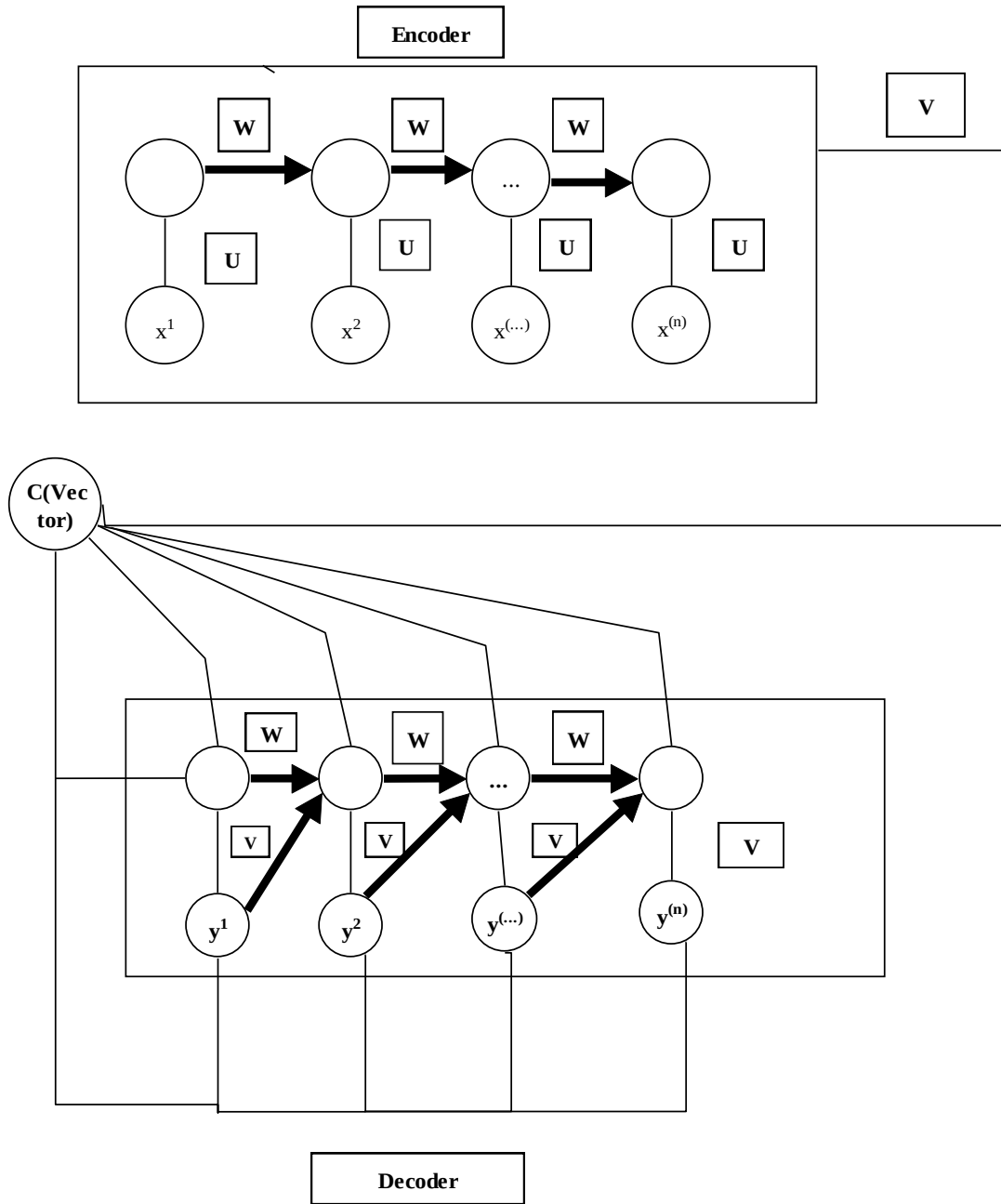


Figure3.2 The forward propagation of encoder-decoder Sequence-to-sequence architecture. The input-to-hidden, hidden-to-hidden, and hidden-to-output weights are denoted by U , W , and V , respectively. The illustration is inspired from Goodfellow et al. (2016). For this thesis the context vector is repeated 4 times so for corresponding 4 context vectors we have 4 decoder setups.

RNN are very similar to MLP that process input to output using hidden layers, however RNN process the input sequentially using the connections across time steps. At each time step t , hidden state $h(t)$ is evolved by processing the input $x(t)$ and also the previous hidden state $h(t-1)$. Therefore, each hidden state $h(t)$ is a function of the previous hidden state and the input as

$$h(t) = f(h(t-1), x(t)) \quad (3.21)$$

Hence, these hidden states can be said to have a kind of *memory* (G'eron, 2019). Also, Goodfellow et al. (2016) argue that $h(t)$ is a lossy summary of past sequences until time t . Furthermore, three different weights are used for connections. Input-to-hidden weights are denoted by U , hidden-to-hidden weights are denoted by W , and hidden-to-output weights are denoted by V . The weight W processes hidden states to obtain the next hidden states, and we refer to these weights as hidden-to-hidden for simplicity. The weight matrices U and W are shared parameters across all time steps (Goodfellow et al., 2016). Goodfellow et al. (2016) argue that shared parameters allow the model to generalize to sequence lengths that do not occur in the training set, hence the estimation of parameters can be carried out with far fewer training samples. Since we use the sequence-to-sequence encoder-decoder architecture with repeated context vectors, the hidden-to-output weights V are present for all the timesteps taken out/forward from the last timestep in encoder layer. This vector is known as context vector C . The connections between inputs, hidden states, and output are demonstrated in Figure 4.2.

In this section, we explain the forward propagation using the vanilla RNN that use simple RNN cells. However, simple RNN cells are very limited in capturing long term dependencies (G'eron, 2019). Therefore, more complex and advanced long short-term memory (LSTM) cells are discussed in Section 4.5. We follow derivations as described in Guo (2013), however the notation is revised for consistency throughout the thesis. Our notation is as follows:

- M : total number of features
- P : total number of hidden states
- K : total number of output classes
- $x_m^{(t)}$: m -th input at time t
- $h_j^{(t)}$: j -th hidden state at time t
- $h_p^{(t-1)}$: p -th hidden state at time $(t - 1)$
- $\hat{y}_k$: k -th output prediction
- b_j : bias term of j -th unit
- u_{mj} : weight from m -th input feature to j -th hidden state
- w_{pj} : weight from p -th previous hidden state to j -th hidden state
- v_{jk} : weight for j -th hidden state to k -th output

The forward propagation of the RNN starts by calculating the activation of the hidden states $a(t) j$ using the input and previous hidden states at each time step t as

$$a_j^{(t)} = \sum_{m=1}^M x_m^{(t)} u_{mj} + \sum_{p=1}^P h_p^{(t-1)} w_{pj} + b_j \quad (3.22)$$

These activations are then processed with an activation function $f(\cdot)$, and hidden states $h(t)$ are obtained as

$$f(a_j^{(t)}) = h_j^{(t)}, \quad \forall j = 1, \dots, P. \quad (3.23)$$

The hidden states are combined with the inputs at each time step and transferred to the next time step. Since we use the encoder-decoder architecture, the output is produced at the every time step $t = t$ for the decoder model. Therefore, the hidden states $h(t)$ at every time- step of the decoder model is used for prediction. The activations for the output layer $o(k)$ are computed using the last hidden states and hidden-to-output weights in the encoder model is given by:

$$o_k = \sum_{p=1}^P h_p^{(\tau)} v_{pk} + b_k, \quad \forall k = 1, \dots, K \quad (3.24)$$

Where τ denotes the last timestep of the last hidden state which is then used to obtain the vector output C .

the output activations are transformed using the output activation function $g(\cdot)$ and context Vectors C are obtained as :

$$g(o_k) = \hat{y}_k, \quad \forall k = 1, \dots, K \quad (3.25)$$

These vectors which can be single or of variable length depending on the number of times the vectors are repeated, are passed on to the decoder model. where for each timestep $t=t$ there is a prediction given.

$$o_k^t = \sum_{p=1}^P h_p^t v_{pk} + b_k, \quad \forall k = 1, \dots, K \quad (3.26)$$

$$g(o_k^t) = \hat{y}_k^t \quad (3.27)$$

Thus, the hidden-to-hidden weights W , input-to-hidden weights U , hidden-to-output weights V , and bias parameters need to be calculated to obtain predictions.

3.3 Backward Propagation

To train the network, the gradient of the loss function L with respect to weights needs to be calculated. Then, the weight update using the gradient descent algorithm can be carried out

$$\begin{aligned} W^{(t+1)} &= W^{(t)} - \eta \frac{1}{N} \sum_{i=1}^N \frac{\partial L_i}{\partial W} \\ U^{(t+1)} &= U^{(t)} - \eta \frac{1}{N} \sum_{i=1}^N \frac{\partial L_i}{\partial U_i} \\ V^{(t+1)} &= V^{(t)} - \eta \frac{1}{N} \sum_{i=1}^N \frac{\partial L_i}{\partial V_i} \end{aligned} \quad (3.28)$$

where L_i represents the loss function of observation i and can be expressed by

$$\frac{\partial L_i}{\partial v_{jk}} = \frac{\partial L_i}{\partial \hat{y}_{ik}} \frac{\partial \hat{y}_{ik}}{\partial o_{ik}} \frac{\partial o_{ik}}{\partial v_{jk}} \quad L_i = -\sum_{k=1}^K y_{ik} \log(\hat{y}_{ik}) \quad (3.29)$$

in case of using cross-entropy loss function. Furthermore, N denotes the total number of observations and η denotes the learning rate. The learning rate can be adjusted rather than being set to a fixed value during training. For instance, adaptive moment estimation (Adam) is one of the most popular optimization algorithms that adjusts the learning rate at each iteration (Kingma and Ba, 2017). The gradient of the loss function with respect to weights depends on lower layers and also across time steps. Hence, the gradient calculations are not straightforward. We need a technique to carry out these gradient calculations. The backpropagation algorithm is used to calculate the gradients using the chain rule recursively for neural networks (Rumelhart et al., 1986). In this section, we derive the gradients only using the last time step of RNN to explain the backpropagation idea. The hidden-to-output weights ($\mathbf{V}$) only appear in the last time step ($t = \tau$), therefore we can directly calculate the gradients. On the other hand, hidden-to-hidden ($\mathbf{W}$) and input-to-hidden weights ($\mathbf{U}$) are used across all time steps ($t = 1, 2, 3, \dots, \tau$). Hence, errors should be backpropagated across all time steps to obtain the complete gradients for $\mathbf{W}$ and $\mathbf{U}$. The application of backpropagation algorithm through time steps is called as backpropagation through time (BPTT) algorithm (Goodfellow et al., 2016; G'eron, 2019). Although we devote this section to explain the main idea of backpropagation and derive the gradients only for the individual time step, the extension of the backpropagation algorithm across time steps is explained in Section 3.4. As in the forward propagation of the RNN, we again follow the derivations of Guo (2013) for backpropagation algorithms. The backpropagation of the errors is illustrated in Figure 3.4.

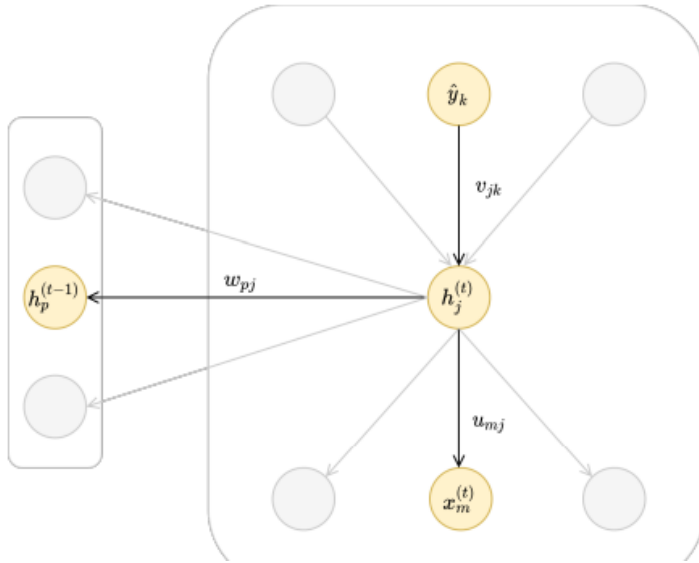


Figure 3.3: Backpropagation of the errors from output to lower layers. The flow also occurs across time steps for RNN.

The gradient of the weights can be obtained by going from output to lower layers. As the first step, the gradient of weight for the j -th hidden state to output k can be computed using the chain rule as

$$\frac{\partial L_i}{\partial v_{jk}} = \frac{\partial L_i}{\partial \hat{y}_{ik}} \frac{\partial \hat{y}_{ik}}{\partial o_{ik}} \frac{\partial o_{ik}}{\partial v_{jk}}. \quad (3.31)$$

. Additionally, we index output layer δ with k , and its equation at time $t = \tau$ is given by

$$\delta_{ik}^{(\tau)} = \frac{\partial L_i}{\partial \hat{y}_{ik}} \frac{\partial \hat{y}_{ik}}{\partial o_{ik}}. \quad (3.32)$$

. The δ allows us to compute the gradients more easily. We will use δ to rewrite the gradient equations in a simpler form. The gradient of the hidden-to-output weight can be rewritten utilizing

layer δ , as plugging Equation 4.1 into Equation 4.2

$$\begin{aligned} \frac{\partial L_i}{\partial v_{jk}} &= \delta_{ik}^{(\tau)} \frac{\partial o_{ik}}{\partial v_{jk}} \\ &= \delta_{ik}^{(\tau)} h_{ij}^{(\tau)}, \end{aligned} \quad (3.33)$$

where $t = \tau$ is the last time step. Therefore, the gradient of weight v_{jk} can be directly computed by multiplying $\delta_{ik}^{(\tau)}$ with the last hidden state j . Since the $h_{ij}^{(\tau)}$ is obtained during the forward propagation, the only missing piece of calculating the gradient is to obtain corresponding δ . Similarly, the last time step gradient of hidden-to-hidden weight is obtained by

$$\frac{\partial L_i}{\partial w_{pj}} = \left(\sum_{k=1}^K \frac{\partial L_i}{\partial \hat{y}_{ik}} \frac{\partial \hat{y}_{ik}}{\partial o_{ik}} \frac{\partial o_{ik}}{\partial h_{ij}^{(\tau)}} \right) \frac{\partial h_{ij}^{(\tau)}}{\partial a_{ij}^{(\tau)}} \frac{\partial a_{ij}^{(\tau)}}{\partial w_{pj}}. \quad (3.34)$$

As we noted at the beginning of this section, the hidden-to-hidden weights are used at each time step. Thus, all time steps must be considered to obtain the complete gradients for hidden-to-hidden weights. We backpropagate the errors across all time steps to obtain the complete gradients in the next session. Now, we continue to obtain the gradients for the last time step. To calculate the gradient for hidden-to-hidden weight, we derive the hidden layer δ . Since we indexed output δ with k , the hidden layer δ is indexed with j , and obtained as

$$\delta_{ij}^{(\tau)} = \left(\sum_{k=1}^K \frac{\partial L_i}{\partial \hat{y}_{ik}} \frac{\partial \hat{y}_{ik}}{\partial o_{ik}} \frac{\partial o_{ik}}{\partial h_{ij}^{(\tau)}} \right) \frac{\partial h_{ij}^{(\tau)}}{\partial a_{ij}^{(\tau)}}. \quad (3.35)$$

Hidden layer δ can be rewritten using output layer δ , as plugging Equation 3.33 into Equation 3.34

$$\begin{aligned} \delta_{ij}^{(\tau)} &= \left(\sum_{k=1}^K \delta_{ik}^{(\tau)} \frac{\partial o_{ik}}{\partial h_{ij}^{(\tau)}} \right) \frac{\partial h_{ij}^{(\tau)}}{\partial a_{ij}^{(\tau)}} \\ &= \left(\sum_{k=1}^K \delta_{ik}^{(\tau)} v_{jk} \right) f'(a_{ij}^{(\tau)}), \end{aligned} \quad (3.36)$$

where k is the index for output layer δ and j for the hidden layer δ . To calculate the gradients of hidden-to-hidden weight using the hidden layer δ , Equation 3.35 is plugged into the Equation 3.36 as

$$\begin{aligned} \frac{\partial L_i}{\partial w_{pj}} &= \delta_{ij}^{(\tau)} \frac{\partial a_{ij}^{(\tau)}}{\partial w_{pj}} \\ &= \delta_{ij}^{(\tau)} h_{ip}^{(\tau-1)}, \end{aligned} \quad (3.37)$$

where $\delta_{ij}^{(\tau)}$ is obtained using the Equation 3.32. Finally, the last time step gradient of input-to-hidden weight can be obtained following the same approach as

$$\frac{\partial L_i}{\partial u_{mj}} = \left(\sum_{k=1}^K \frac{\partial L_i}{\partial \hat{y}_{ik}} \frac{\partial \hat{y}_{ik}}{\partial o_{ik}} \frac{\partial o_{ik}}{\partial h_{ij}^{(\tau)}} \right) \frac{\partial h_{ij}^{(\tau)}}{\partial a_{ij}^{(\tau)}} \frac{\partial a_{ij}^{(\tau)}}{\partial u_{mj}}. \quad (3.38)$$

The gradient of input-to-hidden weight can be rewritten by plugging hidden layer δ as denoted in Equation 4.4 into Equation 4.7

$$\begin{aligned} \frac{\partial L_i}{\partial u_{mj}} &= \delta_{ij}^{(\tau)} \frac{\partial a_{ij}^{(\tau)}}{\partial u_{mj}} \\ &= \delta_{ij}^{(\tau)} x_{im}^{(\tau)}. \end{aligned} \quad (3.39)$$

In this section, we explained the idea of backpropagation that calculates the gradients by applying $\frac{\partial L_i}{\partial w_{pj}} = \sum_{t=1}^{\tau} \delta_{ij}^{(t)} h_{ip}^{(t-1)}$ the chain rule. We obtained expressions $\frac{\partial L_i}{\partial v_{jk}}$, $\frac{\partial L_i}{\partial u_{mj}}$, and $\frac{\partial L_i}{\partial w_{pj}}$. The hidden-to-output weights only appear for every time step in the decoder model, and the obtained expression for $\partial L_i / \partial v_{jk}$ can be directly used to train RNN. However, the gradients of hidden-to-hidden and input-to-hidden weights are not only dependent on the last time step ($t = \tau$) but across all time steps ($t = 1, \dots, \tau$). Consequently, the errors should also be backpropagated across time steps to obtain complete gradients.

3.4 Backward Propagation through time.

Since the input-to-hidden and hidden-to-hidden weights are shared parameters, the errors should be backpropagated across time steps. The gradients for the previous time steps can be computed again using the δ 's as

$$\delta_{ip}^{(t-1)} = \sum_{j=1}^P \delta_{ij}^{(t)} w_{pj} f'(h_{ip}^{(t-1)}) \quad (3.41)$$

where p is the index of the hidden layer at time $t - 1$, and j is the index for hidden layer at time t . Therefore, the gradient for the input-to-hidden weight can be calculated summing across all time steps as

$$\frac{\partial L_i}{\partial u_{mj}} = \sum_{t=1}^{\tau} \delta_{ij}^{(t)} x_{im}^{(t)} \quad (3.42)$$

and for hidden-to-hidden weight as

$$\frac{\partial L_i}{\partial w_{pj}} = \sum_{t=1}^{\tau} \delta_{ij}^{(t)} h_{ip}^{(t-1)} \quad (3.43)$$

3.5 Long short-term memory cells

In RNN, the weights are updated by calculating the gradients across all time steps using the chain rule. Therefore, *vanishing gradients* problem where the gradients get smaller and smaller or *exploding gradients* problem where gradients get larger and larger can occur while errors flow backwards during training (G'eron, 2019). BPTT algorithm can cause exploding gradients that may result to oscillating weights or vanishing gradients that may result to long training times (Hochreiter and Schmidhuber, 1997). Thus, Hochreiter and Schmidhuber (1997) introduced the long short-term memory (LSTM) cells to tackle this problem. Furthermore, G'eron (2019) and Goodfellow et al. (2016) argue that LSTM cells can capture the long-term dependencies of sequences better than simple RNN cells. Cho et al. (2014) also introduced the gated recurrent units (GRU) which is a simplified version of LSTM cells. However, we only discuss the LSTM in this section utilizing the explanations of G'eron (2019). In simple RNN cells, the input is combined with previous hidden states, and the new state emerges after the tanh transformation at each time step as

$$h_p^{(t)} = \tanh \left(\sum_{m=1}^M u_{mp} x_m^{(t)} + \sum_{j=1}^P w_{jp} h_j^{(t-1)} + b_p \right) \quad (3.51)$$

In contrast, the LSTM cells have more complex structure than simple RNN cells as demonstrated in Figure 4.4. LSTM cells can learn which information to store and discard irrelevant ones using the gate structures. LSTM states are divided into two as $h(t)$ and $c(t)$. The $h(t)$ can be considered as short-term state and $c(t)$ as long-term state (G'eron, 2019). To understand how the LSTM cells work, we investigate its functionality part by part. First, p -th long term state $c(t-1)$ subsequently goes through multiplication and addition operations (see top part of Figure 4.4). The first multiplication is called as forget gate and some

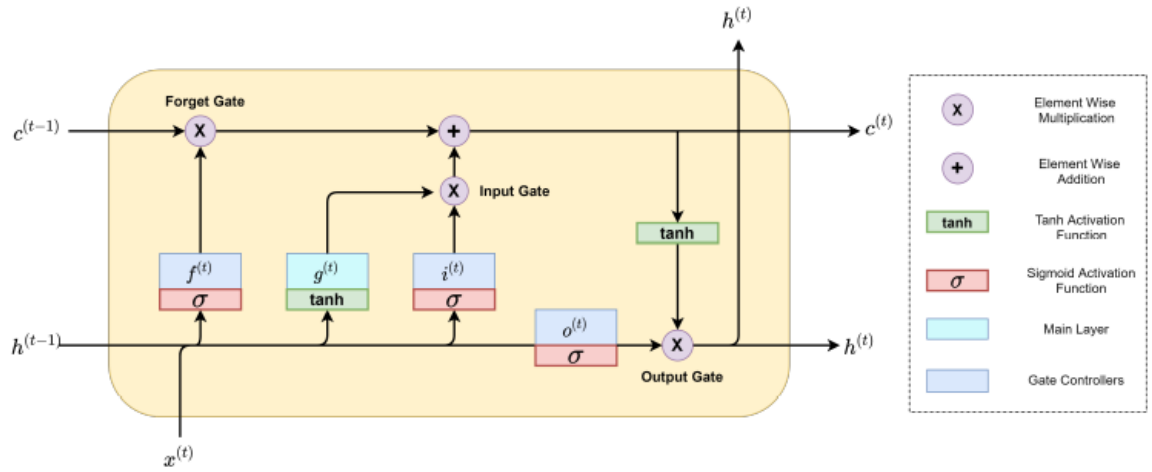


Figure 3.5: Structure of the long short-term memory (LSTM) cells. The illustration is inspired from G'eron (2019)

information in the memory is discarded in this process. Furthermore, the $c(t-1)$ goes through the addition operation where new information is added to the state. After these two operations, $c(t-1)$ is directly transferred to the next time step. Since the forget gate elements are the output of the sigmoid activation function, the state $c(t-1)$ is multiplied with a value that ranges from zero to one. Hence, if the output of the sigmoid function is one, the information is directly transferred. However, the information is deleted if the output of the sigmoid function is zero. Therefore, $f(t)$ (for time step t and state p) is called as forget gate controller and determines if the long-term state $c(t-1)$ will be discarded or not. $f(t)$ controls the forget gate by

$$f_p^{(t)} = \sigma \left(\sum_{m=1}^M u_{mp}^f x_m^{(t)} + \sum_{j=1}^P w_{jp}^f h_j^{(t-1)} + b_p^f \right) \quad (3.52)$$

where σ is the sigmoid activation function and u_{mp}^f , w_{jp}^f , b_p^f are the input-to-hidden weight, hidden-to-hidden weight, and bias for the forget gate, respectively. Next, the short-term state $h(t-1)$ and the feature input $x(t)$ are transferred to the main layer $g(t)$. The simple RNN cells only include the g

(t) p that transforms previous hidden state $h(t-1)$ and $x(t)$ using the tanh activation function. Therefore, $g(t) p$ is responsible for examining the previous short-term states and the feature inputs at the current time step as

$$g_p^{(t)} = \tanh \left(\sum_{m=1}^M u_{mp}^g x_m^{(t)} + \sum_{j=1}^P w_{jp}^g h_j^{(t-1)} + b_p^g \right) \quad (3.53)$$

However, the $g(t) p$ is not directly added to the states. The input gate controller $i(t) p$ decides about information update at the input gate by

$$i_p^{(t)} = \sigma \left(\sum_{m=1}^M u_{mp}^i x_m^{(t)} + \sum_{j=1}^P w_{jp}^i h_j^{(t-1)} + b_p^i \right) \quad (3.54)$$

Since $i(t) p$ includes the sigmoid activation function, the outputs close to one keep the information obtained at main layer $g(t) p$ and discard if outputs are close to zero. Consequently, the old information is dropped and new information is added to the $c(t-1) p$ using the input and forget gate as

$$c_p^{(t)} = f_p^{(t)} c_p^{(t-1)} + g_p^{(t)} i_p^{(t)} \quad (3.56)$$

Last, the output for the time step t should be decided. The output depends on the tanh transformation of the long-term state $c(t-1) p$ and the output gate controller $o(t) p$. Similarly, $o(t) p$ controls the gate with values range from zero to one using sigmoid activation function as

$$o_p^{(t)} = \sigma \left(\sum_{m=1}^M u_{mp}^o x_m^{(t)} + \sum_{j=1}^P w_{jp}^o h_j^{(t-1)} + b_p^o \right) \quad (3.57)$$

Finally, the output and the hidden state that is transferred to the next time step is obtained as

$$h_p^{(t)} = \tanh(c_p^{(t)}) o_p^{(t)} \quad (3.58)$$

3.6 Gated Recurrent Neural Network

In this section we are going to discuss about the theoretical explanation of *The Gated Recurrent Neural Network*(GRU) as discussed by Ian D.Jordan et al(2021).The GRU has two internal gating variables: z_t , which protects the d -dimensional hidden state $h_t \in \mathbb{R}^d$ and the *reset gate* r_t which permits the concealed state to be overwritten and regulates the interaction with the input $x_t \in \mathbb{R}^p$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (3.6.1)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (3.6.2)$$

$$h_t = (1 - z_t) \odot \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) + z_t \odot h_{t-1} \quad (3.6.3)$$

$W_z, W_r, W_h \in \mathbb{R}^{d \times p}$ and $U_z, U_r, U_h \in \mathbb{R}^{d \times d}$ are the parameter matrices, $b_z, b_r, b_h \in \mathbb{R}^d$ are the bias vectors, denotes element-wise multiplication, and $\sigma(\mathbf{z}) = 1/(1+e^{-z})$ is the element-wise logistic sigmoid function. Due to the saturating nonlinearities, the hidden state is asymptotically confined within $[-1, 1]^d$, suggesting that if the state is initiated outside of this trapping zone, it must eventually join it in finite time and remain there for all subsequent time.

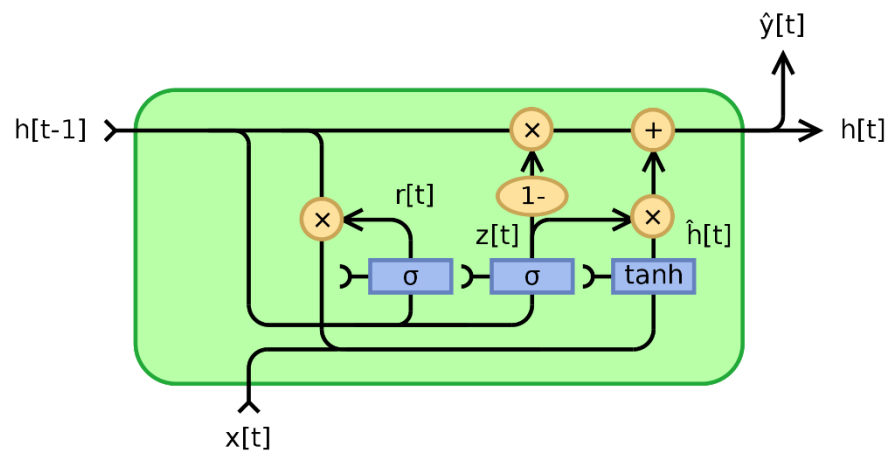


Figure 3.6: Structure of the gated recurrent neural network

4.Methology

Steps for the methodology and implementation of the deep learning algorithm are presented in **Figure 1** and explained further.

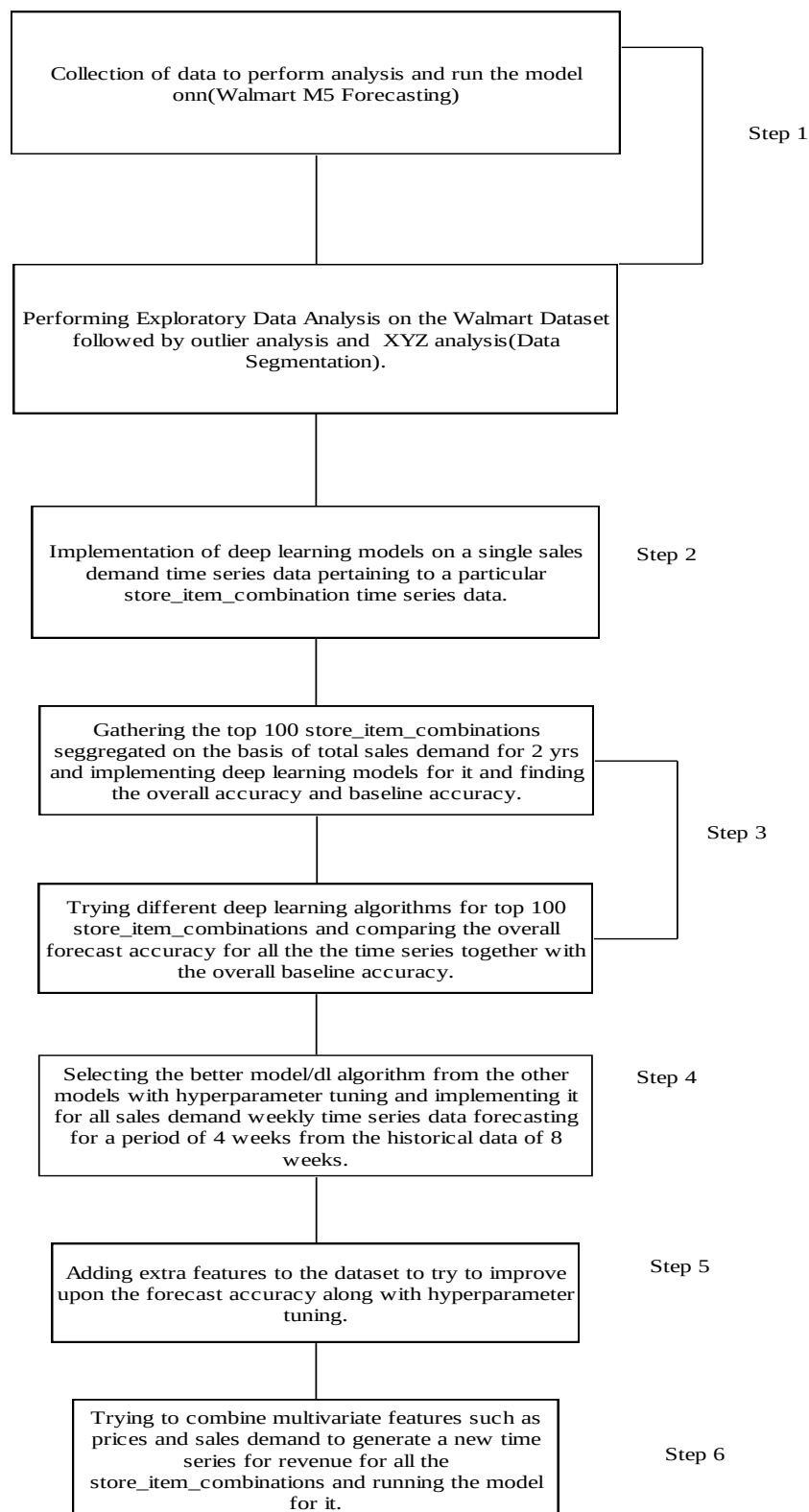


Figure 1. Flow chart for the methodology applied for the implementation of Deep Learning techniques in demand Planning(Problem Statement)

Step 1: Collection of data

In this step, The Walmart sales demand dataset is taken from the popular M-5 forecasting competition hosted on Kaggle. This dataset consists of hierarchical data(data in the form of dataset)containing sales demand and prices for 3 states in the USA(California,Wisconsin,Texas). and includes item level, department, product categories, and store details. In addition, it has explanatory variables such as price, promotions, day of the week, and special events. Together, this robust dataset can be used to improve forecasting accuracy.

Step 1.2:Exploratory data analysis along with XYZ Analysis(Data Segmentation and outlier handling was performed)

To know the data better to make useful insights and inferences ,the data is explored more to know about the data better. The data analysis is performed at an overall level as well for some different levels of hierarchy like at store level ,item level and sales for occasional days.This step is explained in detail later in this section. A particular Recurrent Deep Learning model (Long-Short-Term-memory model) is selected with suitable no of hyperparameters for getting a certain level of accuracy which is Evaluated using MAPE(mean absolute percentage error).(reference for mean absolute percentage error).

Step 2: Implementation of a time series model for a single sales demand store_item_combination time series data

A single univariate sales demand time series is selected for implementation first.This time series data is pertaining to the particular store_item_combination FOODS_1_CA_1_evaluation.the accuracy obtained by the lstm model is compared with the baseline accuracy.

Step 3: Implementation of the deep learning model for multiple time series demand store_item_combination time series data.

This model is then extended for top 100 time series data segregated on the basis of total sales made for 3 consecutive years(2014,2015,2016) and compared with the baseline accuracy.

Step 3.1: Implementation of the different deep learning model for multiple time series salesdemand store_item_combination data.

Different deep learning models are implemented for the top 100 store_item_combinations(skus) and overall accuracies for all the models a re compared with the baseline accuracy.This experiment is done to look for the model which provides better accuracy amongst the others

Step 4:the better model with the suitable hyperparameters is selected from the sample data and implemented for all the store_item_combination_data.

The model selected from experimentation is then extended for implementation for all 30490 store_item_combinations sales demand time series to forecast the sales of next 4 weeks from past 8 weeks of time series data.

Step 5:Adding extra features to the dataset to try to improve upon the forecast accuracy along with suitable hyperparameter tuning

In an attempt to improve upon the forecast accuracy some explanatory variables like the sales on days of special events and on SNAP days are added as extra columns in addition to the sales demand data for every sku. It results in increase of accuracy by a couple of percentages.

Step 6: Trying to expand our predictions for sales demand by using multivariate features like sales demand and prices.

Experimenting on different ways to include multiple attributes(input variables) to predict the future sales demand of next 4 weeks.

The data is first implemented at daily level for a testing horizon of 28 days and then aggregated at weekly level for a testing horizon of 4 weeks.

5. Case Study Analysis

LSTM and MLP algorithms were used to determine the forecast for next 28 days to check for the proper working of the model. These forecasts were then latter aggregated to weekly level for all sku(store item combinations) time series data and then other models like 1-d CNN ,GRU and RNN are used to evaluate the forecast accuracy for all the items. The model with the better accuracy amongst all is selected as the final model for implementation over all the skus. The accuracy is then taken out for all the skus. The time series data from the dataset is prepared by extracting time based features from the data mainly the week and then passing the data through the cycle mentioned below.

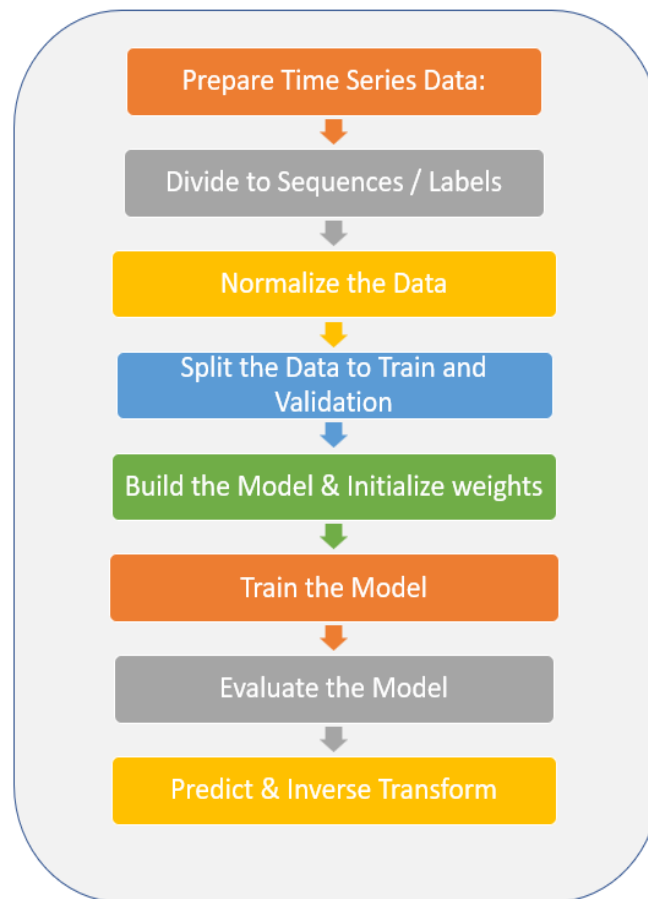


Figure 2:Steps explaining the process of implementation of deep learning models for time series data.

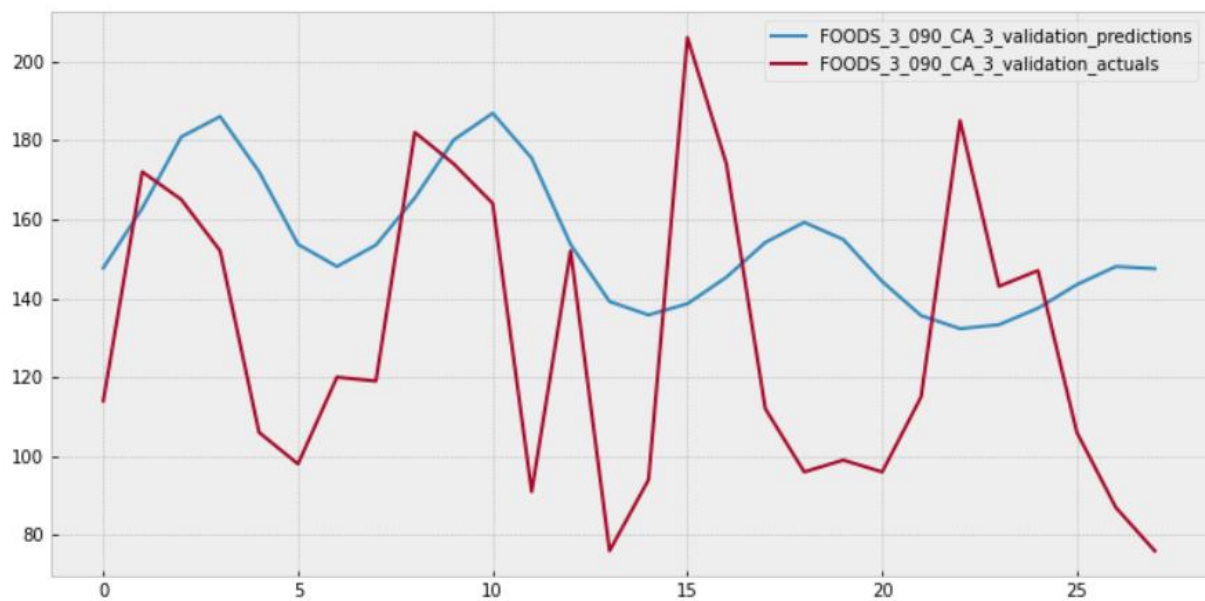


Figure 3:Forecast of 4 weeks(28 days) of prediction horizon using multilayer encoder-decoder LSTM for a particular store_item combination.

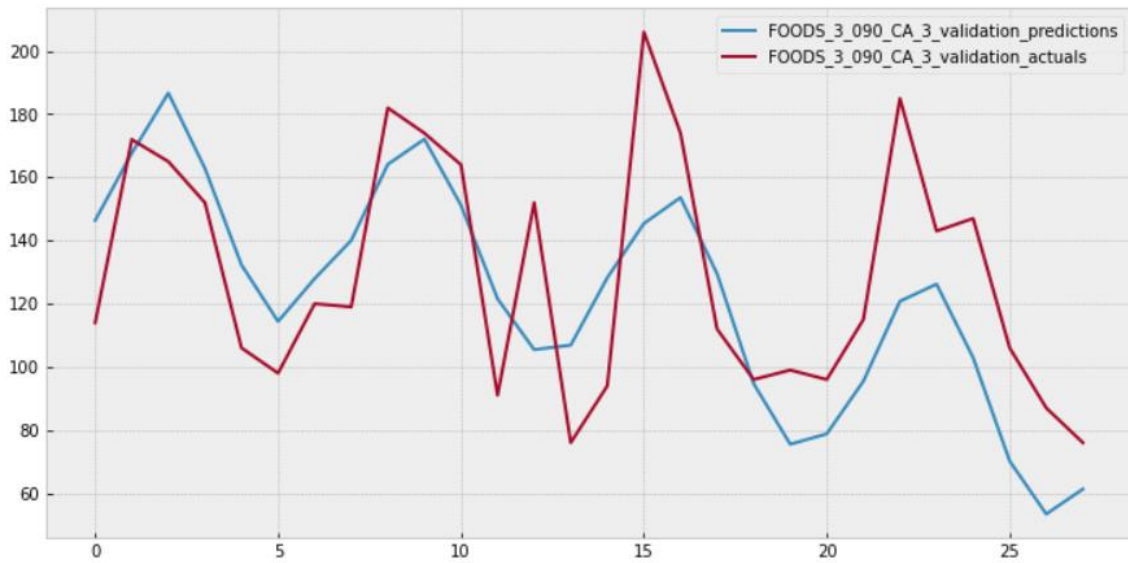


Figure 4. Forecast of 4 weeks of prediction horizon(at day level) using multilayer perceptron for a particular store_item combination.

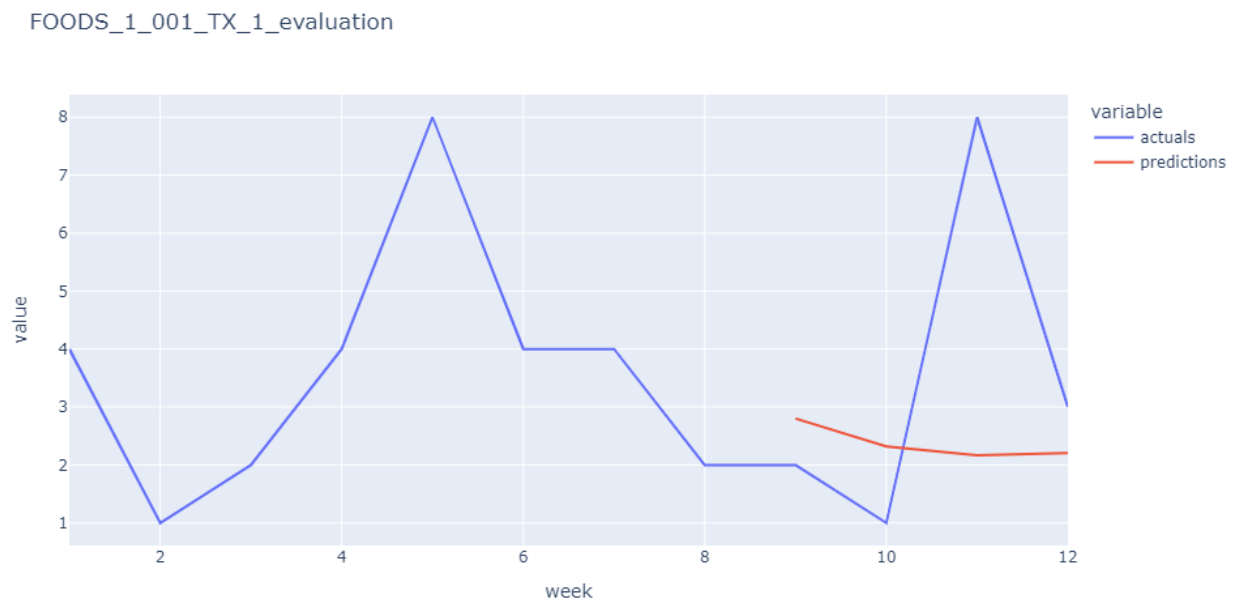
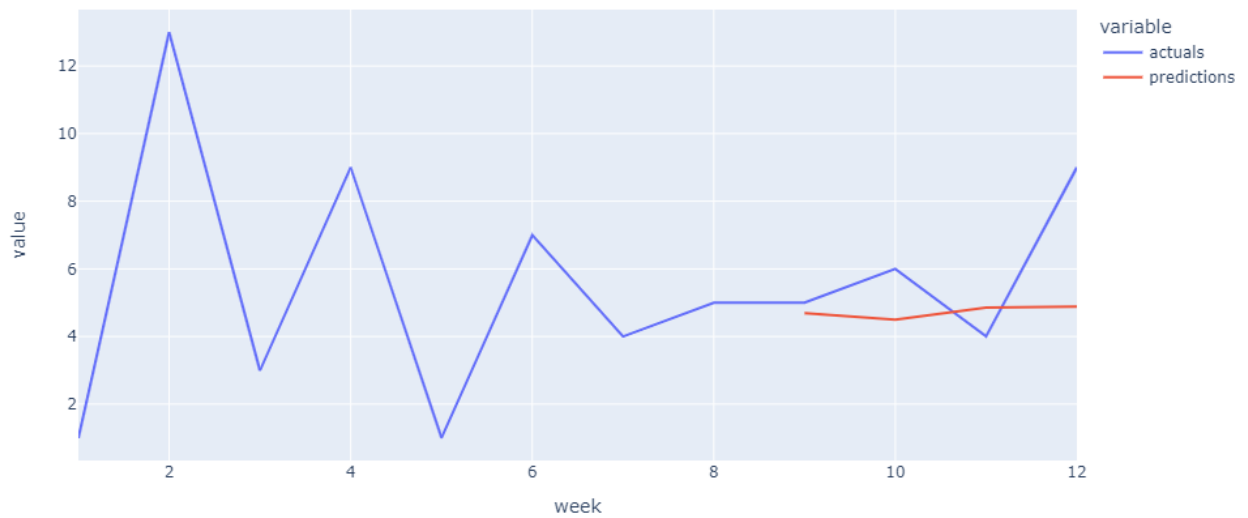


Figure 5.forecasts for sku FOODS_1_001_TX_1_evaluation taken at weekly time horizon.

FOODS_1_001_CA_2_evaluation

**Figure 6.**forecasts for sku FOODS_1_001_CA_2 evaluation taken at weekly level

iteration	Iteration Description	accuracy
1	baseline seq2seq lstm layer with one encoder layer and one decoder layer	78.07% baseline accuracy(weekly level)
2		
3	many to one model(for one sku(testing)) (it had shown no seasonality)	52% accuracy(for one sku-many-to-one model)(daily level)
4	added two lstm layers(hidden layers neurons=128)	64% accuracy(2 layer lstm encoder-decoder)(daily level)78.68% (weekly)
5	added two more layers with hidden layer size (128,128) and added snap days as features	67% accuracy(4-layer lstm encoder-decoder model)(daily level)
6	change the no of timesteps in to 365 days(x(input)) for a prediction horizon of 28 days(y(output))	54%accuracy at daily level(like for historical data of 28 days)
7	changed the model to mlp with 3 layers and no of hidden neurons as 128	67%accuracy(daily-level)(also able to capture seasonality of forecast)79.55%(w
8	implemented mlp with 3 layers and no hidden units 128,64,32 also took the day before events and snap features as input data	
9	implemented mlp with 3 layers and no hidden units 128,128,128 with data at weekly level aggregated data and changing the learning rate to 0.01	79.998% weekly level accuracy
10	implemented mlp with 3 layers and no hidden units 128,128,128 with data at weekly level aggregated data and changing the learning rate to 0.01 and changing loss metrics to huber	80.92%weekly level accuracy
11	implemented mlp with 3 layers and no hidden units 100,100,100with data at weekly level aggregated data and changing the learning rate to 0.01 and changing loss metrics to huber(2nd iter)	81.24% weekly level accuracy
12	implemented lstm with 3 layers and no of hidden units 100,100,100 and changing the learning rate to 0.01 and changing the loss metrics to huber(many-to-many architecture)	82.5% weekly level accuracy
13	implemented lstm with 3 layers and no of hidden units 100,100,100 and changing the learning rate to 0.001 and changing the loss metrics to mse(many-to-many architecture)	81.05% weekly level accuracy
	none	
14	implemented gru with 2 layers and no of hidden units 100,100(encoder-decoder),learning rate 0.001 and changing the loss metrics to huber(encoder-decoder GRU)	80.60% weekly level accuracy
15	implemented gru with 3 layers and no of hidden units 100,100,100(each for encoder-decoder),learning rate 0.01 and changing the loss metrics to huber(encoder-decoder GRU)	82.8% weekly level accuracy(83% approx)
16	implemented 1Dconv neural network for timesteps of 8weeks and prediction horizon of 4 weeks ,filters_used=64,kernel size=2(only a single cnn layer),1 dense layer with hidden units 50,loss='mse')	77.8% weekly level accuracy
17	implemented 1D cnn for timesteps of 8 weeks and prediction hoizon of 4 weeks ,filters_used=64,32,16,16 each with kernel size=3,3,2,2,3 dense layers with hidden units=100,100,100,loss='huber'	74.91% weekly level accuracy(less than naive)(performing relatively poor on c
18	implemented gru with 3 layers and no of hidden units 100,100,100(each for encoder-decoder),learning rate 0.001 and changing the loss metrics to mse(encoder-decoder GRU) for one store-category slice	68% weekly level accuracy(need to improve upon this)(not run the models for
19	implemented gru with 3 layers and no of hidden units 100,100,100(each for encoder-decoder),learning rate 0.01 and changing the loss metrics to huber(encoder-decoder GRU) for all the skus (30,490 skus)	58-60% weekly accuracy(with baseline accuracy as 54.4%(naive))
20	implemented gru with 3 layers and no of hidden units 64,64,64(each for encoder-decoder),learning rate 0.01 and changing the loss metrics to huber(encoder-decoder GRU) for all the skus (30,490 skus)	63% weekly accuracy for 30490 skus
21	implemented my best model for 30490 features with some added features(to time series columns) (snap features)	47.1% accuracy(need to rectify)
22	implemented my best model for 30490 features with some added features(to time series columns) (snap features) (changing hyperparameters to 0.001 and loss function to mse)	46.62% accuracy
23	implemented my best model for 30490 features with some added features(to time series columns) (snap features) (changing hyperparameters to 0.001 and loss function to mse)	59% accuracy
24	implemented my best model for top 100 skus with some added features(to time series columns) (snap features) (changing hyperparameters to 0.001 and loss function to mse)	82.9% accuracy
25	implemented the GRU model taking additional features(snap days) and taking 2 layers of encoder-decoder (128 units each) and changing the no of epochs to 200,batch-size=128,and loss as mse(top 100 skus)	83.11% accuracy
26	implemented the GRU model taking additional features(snap days and events) and taking 2 layers of encoder-decoder (128 units each) and changing the no of epochs to 200,batch-size=128,and loss as mse(all 30490)	63% accuracy(with baseline as 54.4% (naive))
27	changed the input data for (revenue) and applied the same model for sales revenue time series data for all the skus,(combining sales demand and average sell prices)	58-59% accuracy at weekly level(with baseline as 52.03% accuracy)

Table 1.Table for every iteration along with the iteration description and accuracy.

After running the iterations for top 100 skus, predictions for all the deep learning models are taken and the better performing deep learning algorithm is taken as the model for running on all the 30490 skus and the accuracy is noted down.

Table 2. Evaluation for all 30490 skus

		Test Model	BCA
		naïve accuracy	52.2% accuracy
		Accuracy of the GRU model	59% accuracy
		Forecast accuracy after feature addition	62.3% accuracy

25-03-2022	21	implemented my best model for 30490 features with some added features(to time series columns) (snap features)	47.1% accuracy(need to rectify)
26-03-2022	22	implemented my best model for 30490 features with some added features(to time series columns) (snap features) (changing hyperparameters to 0.001 and loss function to mse)	46.62% accuracy
26-03-2022	23	implemented my best model for 30490 features with some added features(to time series columns) (snap features) (changing hyperparameters to 0.001 and loss function to mse)	59% accuracy
27-03-2022	24	implemented my best model for top 100 skus with some added features(to time series columns) (snap features) (changing hyperparameters to 0.001 and loss function to mse)	82.9% accuracy
28-03-2022	25	implemented the GRU model taking additional features(snap days) and taking 2 layers of encoder-decoder (128 units each) and changing the no of epochs to 200, batch-size=128, and loss as mse(top 100 skus)	83.11% accuracy
28-03-2022	26	implemented the GRU model taking additional features(snap days and events) and taking 2 layers of encoder-decoder (128 units each) and changing the no of epochs to 200, batch-size=128, and loss as mse(all 30490)	63% accuracy(with baseline as 54.4% (naive))
29-03-2022	27	changed the input data for (revenue) and applied the same model for sales revenue time series data for all the skus, (combining sales demand and average sell prices)	58-59% accuracy at weekly level(with baseline as 52.03% acc

Table 3. Table for every iteration on the best model for all 30490 store_item combinations on the dataset. The iteration marked in yellow shows the best accuracy obtained of all the iterations.

6. Conclusions and Scope For Future Work

In this report, the work on implementation of the state-of-art deep learning model was discussed in detail for solving Time series forecasting problems on the Walmart hierarchical based datasets. The top 100 skus were taken for experimentation and different types of deep learning models were applied out of which the gated recurrent neural network performed better and implemented on the dataset. Also after the addition of features such occasions and events the model tends to perform better. After the successful implementation on Walmart dataset we have been given some dataset from some big retailers to perform the same procedures as before and forecast the future sales. So, this requires further investigation and work to be done on. Therefore, a further study and work on other unknown multiple time series problem by RNN architectures like LSTM and GRU and other deep learning models need to be done.

References

1. Chopra, S., & Meindl, P. (2007). Supply chain management. Strategy, planning & operation. In *Das summa summarum des management* (pp. 265-275). Gabler.
2. Demand planning :What is and why is it important : Abby Jenkins et al.
3. Blocher, J. D., Mabert, V. A., Soni, A. K., & Venkataramanan, M. A. (2004). Forecasting: Including an Introduction to Forecasting using the SAP R/3 System. Indiana University Kelley School of Business.
4. Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., ... & Zheng, X. (2016). Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467*.
5. Géron, A. (2019). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems*. " O'Reilly Media, Inc."
6. Indiveri, G., Linares-Barranco, B., Hamilton, T. J., Schaik, A. V., Etienne-Cummings, R., Delbruck, T., ... & Boahen, K. (2011). Neuromorphic silicon neuron circuits. *Frontiers in neuroscience*, 5, 73.
7. Guo, J. (2013). Backpropagation through time. Unpublished manuscript, Harbin Institute of Technology. <http://ir.hit.edu.cn/~jguo/docs/notes/bptt.pdf> 5
8. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical learning* (Vol. 112, p. 18). New York: springer.
9. Kotsiantis, S. B., Zaharakis, I., & Pintelas, P. (2007). Supervised machine learning: A review of classification techniques. *Emerging artificial intelligence applications in computer engineering*, 160(1), 3-24.
- 10 Lippmann, R. (1987). An introduction to computing with neural nets. *IEEE Assp magazine*, 4(2), 4-22.
11. Labatut, V., & Cherifi, H. (2011). Evaluation of performance measures for classifiers comparison. *arXiv preprint arXiv:1112.4133*.
12. Souza, C. R. (2010). Kernel functions for machine learning applications. *Creative commons attribution-noncommercial-share alike*, 3(29), 1-1.
13. Rumelhart, G. E., & Hinton, R. J. (1986). Williams, Learning representations by back-propagating errors. *Nature*, 323, 533-536.
14. Rizk, Y., Hajj, N., Mitri, N., & Awad, M. (2019). Deep belief networks and cortical algorithms: A comparative study for supervised classification. *Applied computing and informatics*, 15(2), 81-93.
15. Marc Schwartz and various authors for Perl modules listed in each .pm file.(2019),WriteXLS, Cross-Platform Perl Based R Function to Create Excel 2003 (XLS) and Excel 2007 (XLSX), <https://cran.r-project.org/web/packages/WriteXLS>
16. M5 forecasting - product for Walmart - Machine Learning NESTOR PEREIRA -Neural Network https://www.researchgate.net/publication/351360605_M5_forecasting_-_product_for_Walmart_-_Machine_Learning_NESTOR_PEREIRA_-Neural_Network

<https://www.researchgate.net/project/Neural-Network-Multilayer-Perceptron-Model-and-a-Long-short-term-memory-LSTM-neural-network-to-deal-with-multivariable-applying-to-forecast-problems>

.

THE BEST OPTIMAL HYPERPARAMETERS USED For Encoder Decoder GRU model

Loss Function:MSE

Learning rate:0.001

Batch-size:128

No of hidden neurons per layer:128

No of epochs:200

Total no of layers: 2 layers per encoder and decoder.